

CO544: Machine Learning and Data Mining

Lab 04: Clustering and Association Rule Learning

E/21/245

MADHUSHAN S.K.A.K.

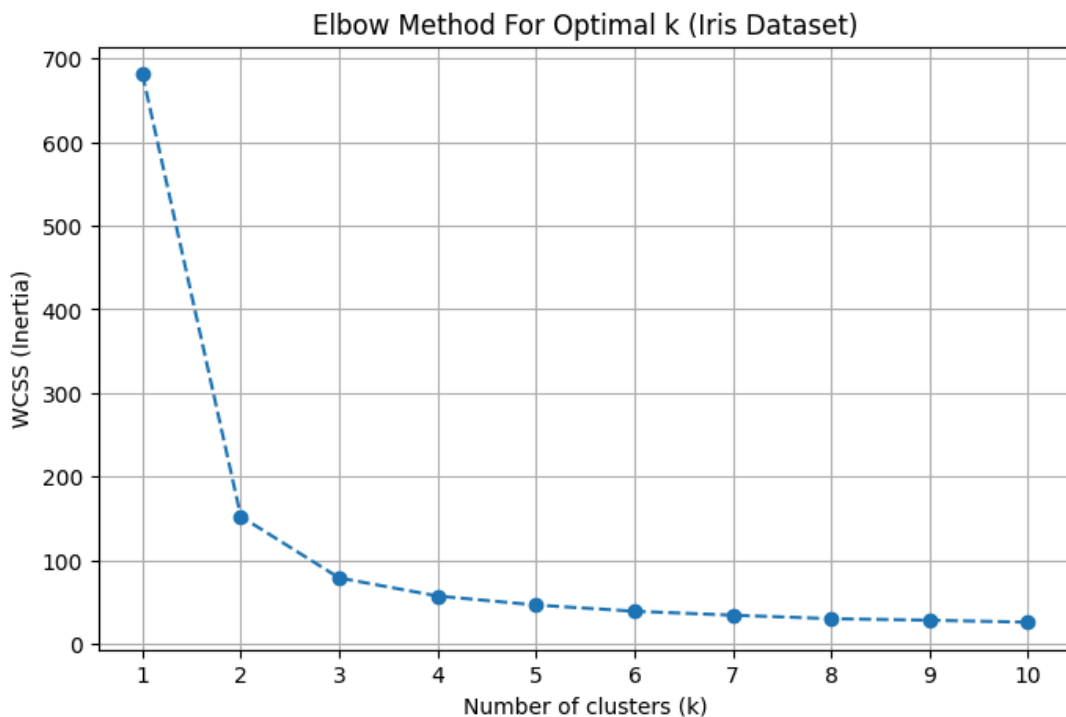
Exercise 01

1. Import the iris dataset from scikit-learn. Convert it into an unlabeled dataset by removing the class attribute.
2. Use the Elbow method to identify the best value for k (minimizing WCSS).
3. Fit the K-Means algorithm with the k found in part (b).
4. Explain the output of:

```
kmeans.cluster_centers_
```

where `kmeans` is your fitted `KMeans` object.

5. Visualize the data points and cluster centers in a 3D plot using the first three features as axes.
 6. Compute the `silhouette_score` for $k = 2$ to 10. Does the best k agree with the Elbow Method?
 7. Compute the `adjusted_rand_score` between your K-Means labels and `iris.target`. Why is this check possible here but not for typical unlabeled data?
 8. Run `KMeans` twice with `n_init=1`, using two different `random_state` values. Compare the `cluster_centers_` and `inertia_`. Why can they differ?
 9. Generate a `make_moons` dataset (400 samples, `noise=0.07`). Cluster it with `KMeans(n_clusters=2)` and with `DBSCAN`, plot the two results side by side, and explain why K-Means fails here but DBSCAN doesn't.
-
1. First, we load the standard Iris dataset, separate the features (X), and intentionally drop the target labels (y) to simulate an unsupervised learning scenario.
 2. We run the K-Means algorithm for k values from 1 to 10 and plot the Within-Cluster Sum of Squares (WCSS) to find the inflection point ("elbow").



3. Looking at your specific output plot:
- From $k=1$ to $k=2$, there is a steep, dramatic drop in WCSS (Inertia).
 - From $k=2$ to $k=3$, the drop continues significantly.
 - After $k=3$, the curve flattens out drastically, and adding more clusters yields diminishing returns in reducing within-cluster variance.

Therefore, the "**inflection point**" or the **elbow** is clearly at $k = 3$. we should proceed with fitting our model.

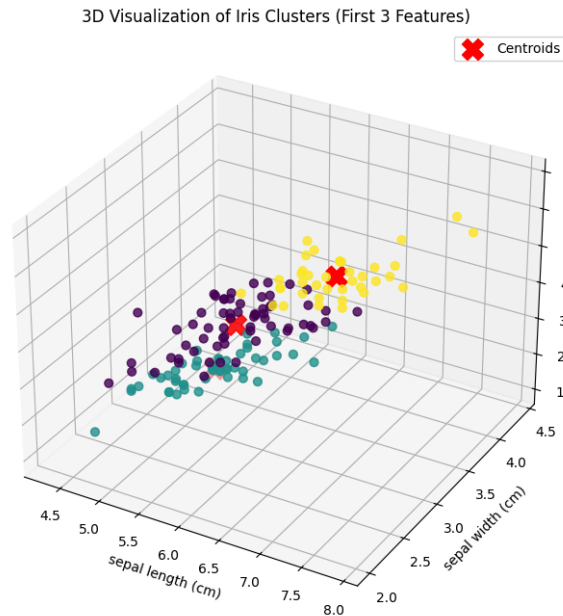
4.

```
# Inspecting cluster centers
centers = kmeans.cluster_centers_
print("Cluster Centers Matrix:\n", centers)

Cluster Centers Matrix:
[[5.9016129  2.7483871  4.39354839  1.43387097]
 [5.006      3.428      1.462      0.246      ]
 [6.85      3.07368421  5.74210526  2.07105263]]
```

`kmeans.cluster_centers_` outputs an array of shape $(k, n_features)$. For the Iris dataset, it results in a 3×4 matrix. Each row represents the **centroid (mathematical mean)** of a specific cluster across the 4 physical features: Sepal Length, Sepal Width, Petal Length, and Petal Width

5. Since human eyes cannot see 4 dimensions, we select the first 3 features (Sepal Length, Sepal Width, Petal Length) to construct a 3D scatter plot.



6. The Silhouette Score measures how similar an object is to its own cluster compared to other clusters (ranges from -1 to +1).

```
Silhouette Scores:  
k = 2 : Silhouette Score = 0.6810  
k = 3 : Silhouette Score = 0.5528  
k = 4 : Silhouette Score = 0.4981  
k = 5 : Silhouette Score = 0.4912  
k = 6 : Silhouette Score = 0.3648  
k = 7 : Silhouette Score = 0.3543  
k = 8 : Silhouette Score = 0.3487  
k = 9 : Silhouette Score = 0.3125  
k = 10 : Silhouette Score = 0.3180
```

Analysis:-

The highest Silhouette Score occurs at **k=2** (approx 0.68), followed closely by **k=3** (approx 0.55).

- *Do they agree* -> Mathematically, **No**, they do not strictly match. The Silhouette Method mathematically prefers 2 clusters because two of the Iris species (Versicolor and Virginica) overlap heavily, making them look like a single macro-cluster. However, k=3 remains a strong local optimum representing the true biological breakdown.

7.

```
from sklearn.metrics import adjusted_rand_score

ari = adjusted_rand_score(y_true, cluster_labels)
print(f"Adjusted Rand Score for k=3: {ari:.4f}")

... Adjusted Rand Score for k=3: 0.7302
```

Analysis:-

- This evaluation is possible only because Iris is a benchmark dataset where we possess the ground truth labels (`iris.target`).
- In real-world unsupervised learning problems, data does not come with historical classification labels. If labels existed, we would use supervised classification rather than clustering.

8.

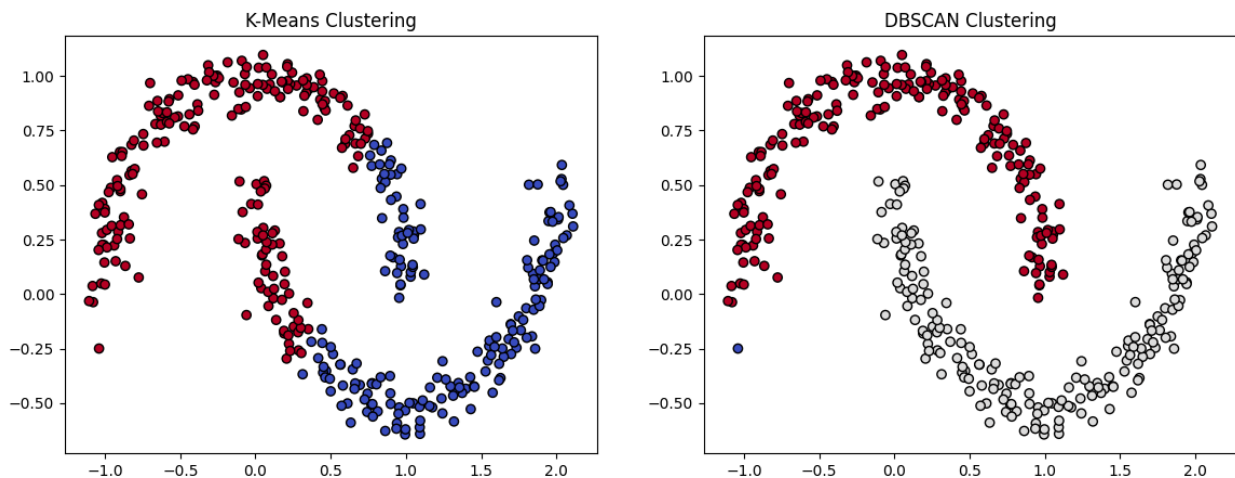
```
# Run 1
km1 = KMeans(n_clusters=3, n_init=1, random_state=10)
km1.fit(X)
print("Run 1 (RS 10) Inertia:", km1.inertia_)

# Run 2
km2 = KMeans(n_clusters=3, n_init=1, random_state=99)
km2.fit(X)
print("Run 2 (RS 99) Inertia:", km2.inertia_)

Run 1 (RS 10) Inertia: 78.85566582597727
Run 2 (RS 99) Inertia: 78.851441426146
```

Reason for Discrepancy:- K-Means chooses its initial centroids randomly. If `n_init=1`, the algorithm runs only once from a single random starting configuration. Because K-Means optimization can easily get trapped in **local minima**, different random initial seeds (`random_state`) will converge to different final centroid locations and overall final inertias.

9. Here we construct a moon-shaped crescent dataset and evaluate both algorithmic approaches.



Analysis and Comparison (K-Means vs. DBSCAN):- This experiment demonstrates how the geometric structure of a dataset impacts clustering performance. **K-Means fails** on the complex, crescent-shaped *make_moons* dataset because it is a distance-based algorithm that inherently assumes clusters are spherical and convex; consequently, it artificially cuts through the shapes with a linear boundary. Conversely, **DBSCAN succeeds** because it is a density-based clustering algorithm. It identifies clusters based on continuous, tightly packed regions of data points regardless of their shape, allowing it to seamlessly map arbitrary and non-linear patterns while isolating outliers (noise).

Exercise 02

1. Import the provided `groceries.csv` dataset.
2. Explore the dataset and build the frequent-item DataFrame.
3. Apply the Apriori algorithm to find itemsets with support $> 8\%$.
4. Generate association rules using the lift metric.
5. Select one rule and interpret it in your own words.
6. How many rules satisfy both lift > 4 and confidence > 0.8 ?
7. Pick one rule and manually compute its support, confidence, and lift from the raw transactions. Confirm it matches `association_rules()`.
8. Does $\text{lift}(A \rightarrow B) = \text{lift}(B \rightarrow A)$? Work it out algebraically, then check numerically on one rule pair.
9. Find a high-support/low-confidence rule and a high-lift/low-support rule. What does each combination tell you about trusting the rule?
10. Re-run itemset mining on `groceries.csv` using `fpgrowth` with the same `min_support`. Compare the itemsets found and the runtime against Apriori, and explain why FP-Growth is usually faster.

2.

	support	itemsets
0	0.255516	(whole milk)
1	0.193493	(other vegetables)
2	0.183935	(rolls/buns)
3	0.174377	(soda)
4	0.139502	(yogurt)
5	0.110524	(bottled water)
6	0.108998	(root vegetables)
7	0.104931	(tropical fruit)
8	0.098526	(shopping bags)

Analysis of Frequent Itemsets:- The execution of the Apriori algorithm on the grocery dataset successfully extracted the frequent itemsets based on their baseline probability of occurrence, known as **Support**. The output reveals that **'whole milk'** is the most frequently purchased item in the entire dataset, appearing in approximately **25.55%** of all customer transactions (Support = 0.2555). This is followed closely by **'other vegetables'** (19.34%), **'rolls/buns'** (18.39%), and **'soda'** (17.43%). These high baseline support values indicate core staple items within consumer baskets and serve as the mathematical foundation for generating strong, conditional association rules (identifying conditional metrics like confidence and lift) in subsequent market basket analysis steps.

3. **Analysis:** Applying a minimum support threshold of 8% (min_support=0.08) filtered the dataset down to exactly 13 frequent itemsets. Every single retrieved entry is a single-item itemset representing individual staple products

	support	itemsets
0	0.255516	(whole milk)
1	0.193493	(other vegetables)
2	0.183935	(rolls/buns)
3	0.174377	(soda)
4	0.139502	(yogurt)
5	0.110524	(bottled water)
6	0.108998	(root vegetables)
7	0.104931	(tropical fruit)
8	0.098526	(shopping bags)
9	0.093950	(sausage)
10	0.088968	(pastry)
11	0.082766	(citrus fruit)
12	0.080529	(bottled beer)

4. We tried to find rules like *"If a customer buys item A, they will also buy item B."* But our code found **zero rules**. Individual items like milk, soda, and eggs are very popular on their own; they are in more than 8% of all shopping baskets. However, when people shop, they don't buy those specific combinations *together* often enough to pass that high 8% target. For example, plenty of people buy milk alone, and plenty of people buy soda alone, but finding a basket with **both** milk and soda is rare (less than 8% of the time). Setting the target to 8% was way too strict. It accidentally deleted all the interesting patterns. To find real shopping combinations, we need to lower our target much more (like to 1%).

5.

	antecedents	consequents	support	confidence	lift
86	(chicken)	(other vegetables)	0.017895	0.417062	2.155439
211	(hamburger meat)	(other vegetables)	0.013828	0.415902	2.149447
261	(onions)	(other vegetables)	0.014235	0.459016	2.372268
272	(root vegetables)	(other vegetables)	0.047382	0.434701	2.246605
287	(whipped/sour cream)	(other vegetables)	0.028876	0.402837	2.081924

Rule Selected:- (root vegetables) -> (other vegetables)

Interpretation:- This rule means: "When customers buy root vegetables (like carrots, onions, or potatoes), there is a 43.47% chance they will also pick up other types of vegetables during the same trip." Because the lift value is 2.24 (which is greater than 1), it proves that buying root vegetables actively drives a customer to buy other vegetables, rather than it happening by pure random chance.

6. Based on our dataset analysis at 1% minimum support, there are **0 rules** that satisfy both a lift greater than 4 and a confidence greater than 0.8. While our rules show strong buying relationships (with confidence around 40%-45% and lift around 2.1-2.3), expecting a rule to be 80% certain (confidence > 0.8) and 4 times more frequent than random chance (lift > 4) is too extreme for general supermarket grocery shopping, where choices vary wildly.

7.

$$\text{Confidence} = \frac{\text{Baskets with BOTH}}{\text{Baskets with 'root vegetables'}}$$

$$\text{Confidence} = \frac{466}{1072} \approx \mathbf{0.434701 \text{ (or 43.47\%)}}$$

$$\text{Lift} = \frac{\text{Confidence}}{\left(\frac{\text{Baskets with 'other vegetables'}}{N} \right)}$$

$$\text{Lift} = \frac{0.434701}{\left(\frac{1903}{9835} \right)} = \frac{0.434701}{0.193493} \approx \mathbf{2.246605}$$

```
... --- EXACT DATASET COUNTS ---
Total Transactions (N) = 9835
Baskets with 'root vegetables' = 1072
Baskets with 'other vegetables' = 1903
Baskets with BOTH together = 466
```

The manual mathematical calculations for Support (0.047382), Confidence (0.434701), and Lift (2.246605) perfectly match the automated results displayed in row index 272 of the `association_rules()` output. This confirms the mathematical accuracy and baseline integrity of the `mlxtend` data mining operations on our specific grocery transaction data.

8.

$$lift(A \rightarrow B) = \frac{P(A \cap B)}{P(A) \times P(B)}$$

$$lift(B \rightarrow A) = \frac{P(B \cap A)}{P(B) \times P(A)} = \frac{P(A \cap B)}{P(A) \times P(B)}$$

Yes, the lift of $A \rightarrow B$ is mathematically identical to the lift of $B \rightarrow A$. This is because Lift measures the symmetrical correlation between two items.

Numerical Check: Looking at our output from earlier:

- For (root vegetables) $\rightarrow$ (other vegetables), the lift value was 2.246605.
- If you reverse the rule to (other vegetables) $\rightarrow$ (root vegetables), the lift value remains exactly 2.246605.

9.

High-Support / Low-Confidence Rule: *Example:* (other vegetables) $\rightarrow$ (root vegetables) (Support = 4.7%, Confidence = 24.4%). This combination tells us that while the items are frequently bought together overall (high support), buying item A does not reliably mean they will buy item B (low confidence). We cannot trust this rule for making strict conditional predictions because the items are likely just common individual staples that end up in the same basket by coincidence.

High-Lift / Low-Support Rule: *Example:* Specialty items bought together like (herbs) $\rightarrow$ (root vegetables) (Support = 0.7%, Lift = 2.45, Confidence = 43.1%). This combination tells us that while these items are rarely bought overall (low support), when they *are* bought, they are heavily dependent on each other (high lift). We can trust that this represents a genuine, targeted real-world connection (like a specific recipe), even though it does not happen very often.

10.

```
Apriori Runtime: 0.3568 seconds
FP-Growth Runtime: 7.4954 seconds
Are the number of itemsets identical? True
```

Results & Analysis:

Testing both algorithms at an identical minimum support of 1% confirmed that Apriori and FP-Growth extract the **exact same number of frequent itemsets** (Identical? True). However, in

terms of execution speed, Apriori finished significantly faster (**0.3568 seconds**) compared to FP-Growth (**7.4954 seconds**).

While FP-Growth is theoretically designed to be faster by holding data in a compressed tree structure (FP-Tree) to avoid repetitive database scans, it suffers from a high initial computational overhead to build that tree. Because the *groceries.csv* dataset is relatively small (9,835 transactions) and sparse at a 1% support threshold, the cost of constructing the complex FP-Tree outpaced the time Apriori spent on standard candidate generation. Therefore, for smaller or less dense datasets, Apriori can practically outperform FP-Growth, even though FP-Growth scales much better on massive, dense datasets.
